

Documentazione Progetto

0. Sommario

1.	Documento di Specifica dei Requisiti Software	2
1.1.	Glossario	2
1.2.	Modellazione dei Casi d'Uso	4
1.3.	Utenti	5
1.4.	Descrizione Requisiti Non Funzionali	5
1.4.1.	Sicurezza	5
1.4.2.	Manutenibilità	6
1.4.3.	Prestazioni	6
1.4.4.	Usabilità	6
1.4.5.	Affidabilità	6
1.5.	Descrizione Requisiti di Sistema	6
1.6.	Descrizione Formale degli Use-Case	7
1.6.1.	Formalismo Tabellare (A. Cockburn)	7
1.6.2.	Mock-Up	10
2.	Documento di Design del Sistema	10
2.1.	Architettura Backend	10
2.1.1.	Java	11
2.1.2.	Spring Boot	11
2.1.3.	PostgreSQL	11
2.1.4.	Spring Data JPA, Hibernate, Specification API	11
2.1.5.	Lombok	12
2.1.6.	Rest API	13
2.1.7.	Spring Security	13
2.1.8.	JWT	13
2.2.	Architettura Front-end	14
2.2.1.	TypeScript	14
2.2.2.	React + React Router	14
2.2.3.	Vite	15
3.	Documento di Design del Software	15
3.1.	Link GitHub	15

3.2.	Schema per la Persistenza dei Dati	16
3.3.	Design Patterns	16
3.3.1.	Dependency Injection	16
3.3.2.	Altri Pattern	16
4.	Attività di Testing	17
4.1.	Tecnologie Utilizzate.....	17
4.1.1.	JUnit	17
4.1.2.	Mockito	17
4.1.3.	AssertJ	18
4.2.	Test e Classi di Equivalenza	19
4.2.1.	createUser(createUserRequest request)	19
4.2.2.	createComment (Long id, CreateCommentRequest request)	20
4.2.3.	changelssue(Long id, ChangelssueRequest request)	21
4.3.	Limiti Individuati e Possibili Estensioni	23

1. Documento di Specifica dei Requisiti Software

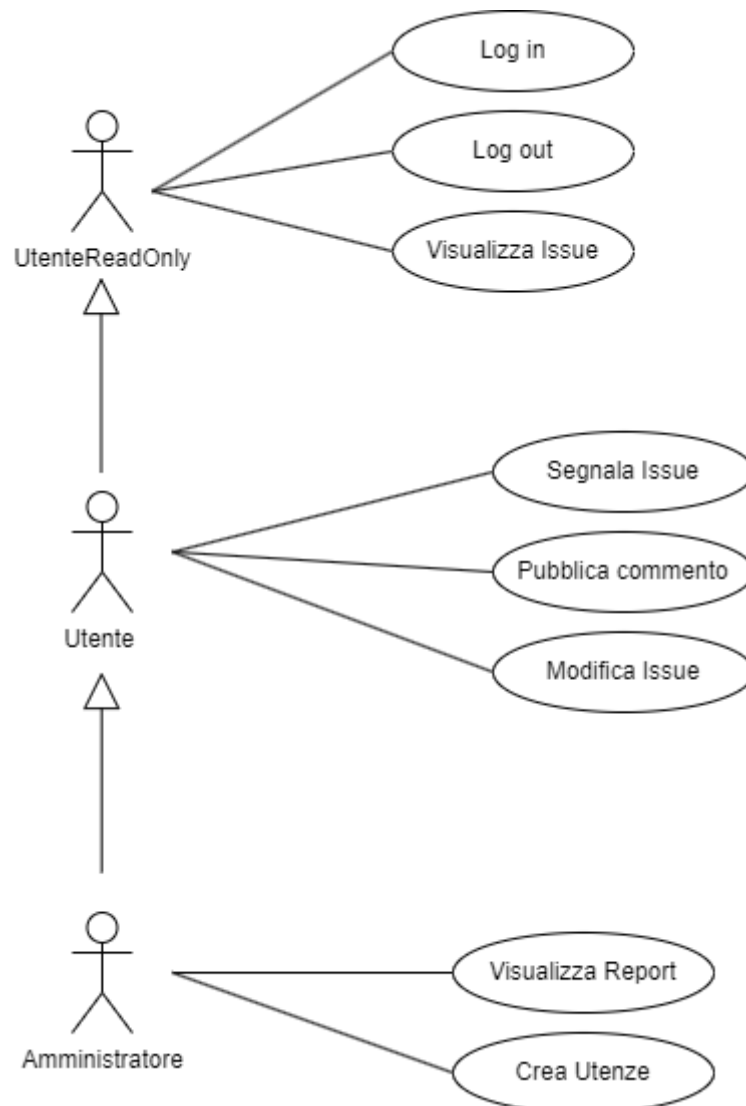
1.1. Glossario

Utente	Qualunque individuo, interno o esterno, che dispone di credenziali di accesso e può autenticarsi nel sistema.
Account	Insieme di credenziali (email e password) che identificano univocamente un utente all'interno del sistema.
Autenticazione	Procedura mediante la quale un utente accede al sistema, fornendo le proprie credenziali (email e password) per verificarne l'identità.
Sistema distribuito	Architettura software composta da più componenti indipendenti (ad esempio, front-end e back-end) che comunicano tra loro per fornire un servizio integrato.

<i>Back-end</i>	Componente server-side del sistema, responsabile della logica applicativa, della gestione dei dati, della sicurezza e della comunicazione con il front-end.
<i>Front-end</i>	Componente client-side del sistema, responsabile dell'interfaccia utente e dell'interazione diretta con l'utente attraverso un'applicazione web, desktop o mobile.
<i>Issue</i>	Segnalazione registrata nel sistema per documentare problemi, richieste di chiarimento o nuove funzionalità. Ogni issue possiede un titolo, una descrizione, uno stato (es. <i>todo</i>), e può avere una priorità e tipologia.
<i>Modalità read-only</i>	Modalità di accesso limitato al sistema che consente esclusivamente la visualizzazione dei dati (issue e commenti), senza possibilità di modifica o inserimento di nuove informazioni.
<i>Commento</i>	Messaggio testuale associato a una issue, utilizzato dagli utenti interni per condividere aggiornamenti, chiarimenti o comunicazioni sullo stato del problema.
<i>Requisiti non-funzionali</i>	Specifiche che descrivono vincoli o caratteristiche qualitative del sistema, come prestazioni, sicurezza, architettura, linguaggi di sviluppo o modalità d'uso.
<i>Requisiti di sistema</i>	Specifiche che definiscono le funzionalità che il sistema deve implementare per garantire il corretto svolgimento delle attività previste.
<i>Stakeholder</i>	Soggetto esterno (cliente, supervisore, partner, ecc.) che ha interesse nel progetto e può accedere al sistema in modalità di sola lettura per monitorare l'avanzamento o lo stato delle issue.
<i>Codice boilerplate</i>	Codice ripetitivo presente nella maggior parte di applicazioni

API (Application Programming Interface)	Letteralmente, un' "interfaccia di programmazione di un'applicazione", è un insieme di regole che permette a diversi software o programmi di comunicare tra loro e scambiarsi dati.
Stateless (senza stato)	Descrive un sistema in cui il server non conserva alcuna informazione sulle richieste precedenti di un client tra una chiamata e l'altra. Ogni richiesta HTTP deve contenere tutte le informazioni necessarie per essere elaborata autonomamente, senza fare affidamento su uno stato salvato lato server come una sessione.

1.2. Modellazione dei Casi d'Uso



1.3. Utenti

Il progetto è stato realizzato suddividendo gli utenti che interagiscono con il sistema in tre diverse categorie:

- 1) **UtenteReadOnly**: rappresenta uno stakeholder o un utente esterno che non fa parte del team di sviluppo, ma ha interesse nel loro lavoro e deve quindi poter accedere al sistema e verificare il progresso dei lavori ed eventuali issue o problemi rilevati. L'utente non è in grado di segnalare issue, modificarle e neanche commentare: si limita a visualizzare i dati presentati
- 2) **Utente**: rappresenta un effettivo membro del team di sviluppo. È in grado di accedere al sistema e visualizzare issue, ma ha anche ulteriori permessi rispetto ad un utente read-only. Tra cui il poter segnalare issue e problemi riscontrati, modificare la issue a cui è stato assegnato e pubblicare commenti su qualunque issue, anche se non è stato assegnato ad essa.
- 3) **Amministratore**: rappresenta un membro del team di sviluppo che supervisiona i progressi. È in grado di creare utenze (normali, read-only o da amministratore), visualizzare report mensili generati automaticamente. Inoltre gode di tutti i permessi delle utenze precedentemente elencate: può quindi visualizzare le issue, modificarle (anche se non assegnato ad esse), pubblicare commenti.

1.4. Descrizione Requisiti Non Funzionali

I requisiti non funzionali definiscono le qualità del sistema, cioè come il sistema deve svolgere le sue funzioni. Questi sono:

1.4.1. Sicurezza

- 1) **Autenticazione e Autorizzazione**: Il sistema deve proteggere le risorse riservate tramite autenticazione sicura (token JWT)
- 2) **Integrità e Segretezza dei Dati**: Le password devono essere salvate nel database e gestite con algoritmi di hashing sicuri (BCrypt)
- 3) **Controllo degli accessi basato sui ruoli**:
 - 3.1) Gli utenti tradizionali possono modificare solo le issue a loro assegnate.
 - 3.2) Gli utenti read-only possono esclusivamente visualizzare i dati senza invocare API di modifica/scrittura.
 - 3.3) Gli amministratori hanno accesso completo a tutte le risorse e alle metriche.

1.4.2. Manutenibilità

- 1) **Qualità del Codice:** Il codice Back-end deve rispettare gli standard di qualità verificabili tramite SonarQube

1.4.3. Prestazioni

- 1) **Generazione Report:** Il calcolo delle metriche mensili per la reportistica amministrativa deve essere ottimizzato a livello di query/database per evitare rallentamenti generali.

1.4.4. Usabilità

- 1) **Intuitività:** L'interfaccia Front-end deve consentire a un utente di segnalare un bug o inserire un commento in massimo 3-4 click.

1.4.5. Affidabilità

- 1) **Consistenza dei Dati:** Le transazioni su database (es. cambio di stato di un bug, aggiunta di un commento) devono garantire le proprietà ACID:
 - 1.1) **Atomicità:** una transazione è un'unità indivisibile. O vengono eseguite tutte le operazioni al suo interno, oppure non ne viene eseguita nessuna; se una parte fallisce, l'intera operazione viene annullata (*rollback*).
 - 1.2) **Coerenza:** ogni transazione porta il database da uno stato valido a un altro, rispettando tutte le regole, i vincoli di integrità e le specifiche di dominio stabilite.
 - 1.3) **Isolamento:** più transazioni eseguite in contemporanea non devono interferire tra loro. Il risultato di transazioni simultanee deve essere identico a un'esecuzione sequenziale.
 - 1.4) **Durabilità:** una volta che una transazione è stata completata con successo (*commit*), le modifiche diventano permanenti e non vanno perse nemmeno in caso di guasto o interruzione di corrente del sistema
- 2) **Tolleranza ai Guasti:** In caso di errore lato server, il Back-end deve restituire codici di stato HTTP appropriati.

1.5. Descrizione Requisiti di Sistema

I requisiti di sistema descrivono cosa il sistema deve fare (a differenza dei non-funzionali, che descrivono come/quanto bene lo fa). Dalla traccia si ha che:

1. Deve essere implementato un sistema di autenticazione semplice e sicuro, basato su email e password. Le informazioni gestite dall'applicazione sono critiche per l'azienda, ed è fondamentale preservarne l'integrità e la segretezza. Il sistema viene fornito con un account

da amministratore già attivo, con credenziali di default. Un amministratore può creare ulteriori utenze, specificando una email, una password, e indicando se quell'utenza sarà "normale" oppure "di amministrazione".

2. Tutti gli utenti autenticati possono segnalare una issue indicando almeno un titolo e una descrizione. Alcuni utenti potrebbero voler specificare anche una priorità e sarebbe gradita la possibilità di allegare un'immagine. Le issue possono essere di diverso tipo: question (per richieste di chiarimenti), bug (per segnalare malfunzionamenti), documentation (per segnalare problemi relativi alla documentazione), e feature (per indicare la richiesta o il suggerimento di nuove funzionalità). Le issue create sono inizialmente nello stato "todo".

3. Il sistema deve offrire una vista riepilogativa delle issue, con la possibilità di filtrare o ordinare i risultati in base a criteri come tipologia, stato, priorità o altri parametri rilevanti.

5. Deve essere prevista una sezione per commenti associata a ciascun bug, in cui tutti gli utenti possano lasciare messaggi, aggiornamenti o richieste di chiarimento.

9. Gli utenti devono poter modificare solo i bug a loro assegnati, mentre gli amministratori devono avere accesso completo.

15. Il sistema deve offrire una modalità "readonly" per utenti esterni (es. stakeholder), che consenta di visualizzare i bug e i commenti senza possibilità di modifica. Gli account per questi utenti vengono creati dagli amministratori.

17. Gli amministratori devono poter visualizzare report mensili sull'attività del team, contenenti metriche (numero di bug aperti/gestiti, tempo di risoluzione medio, etc) aggregate e dettagliate per singolo utente.

1.6. Descrizione Formale degli Use-Case

Tra gli use-case del progetto, i due scelti per essere approfonditi formalmente sono la creazione di un commento da parte di un utente autenticato e la visualizzazione delle issue.

1.6.1. Formalismo Tabellare (A. Cockburn)

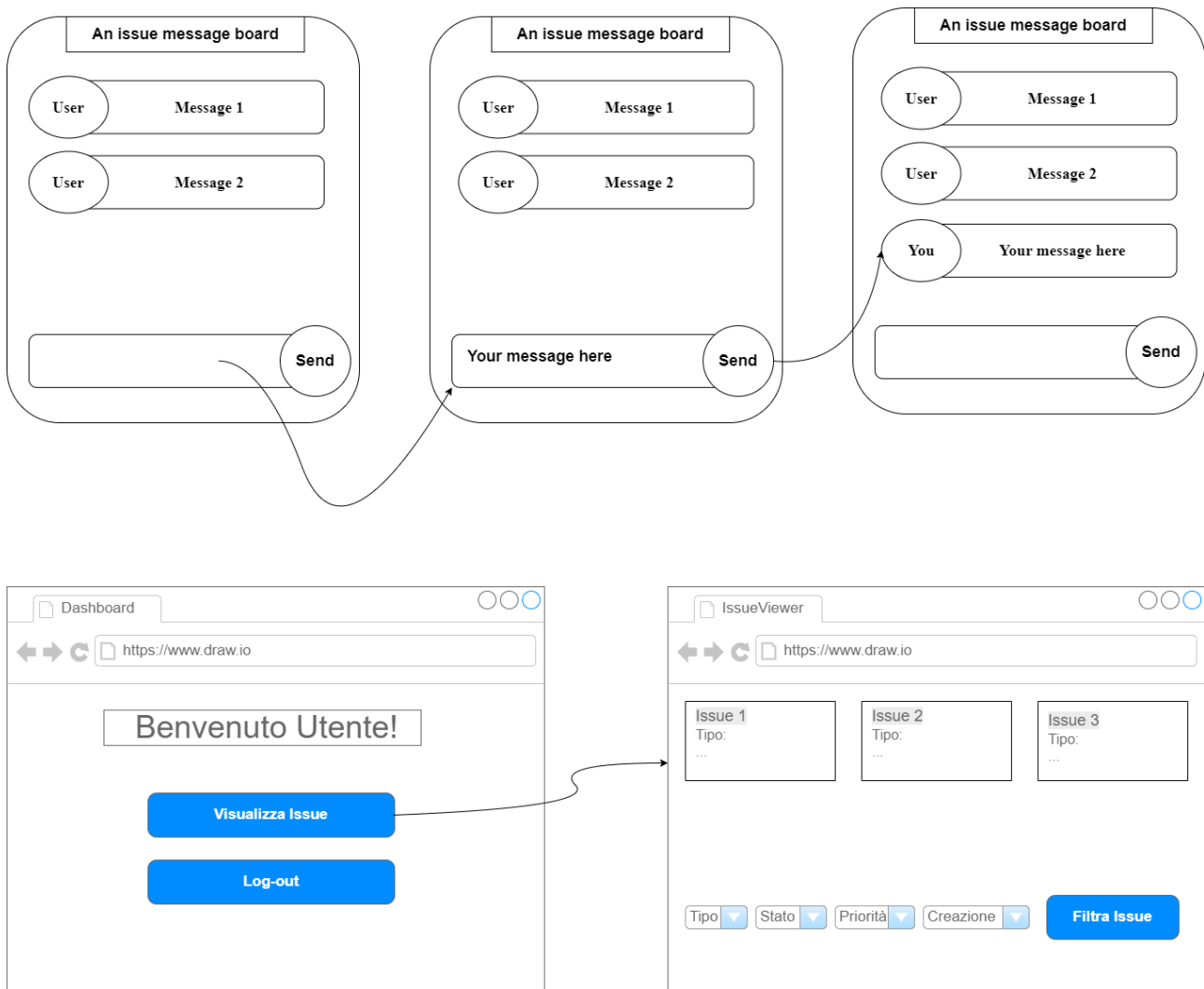
USE CASE #1	Pubblica commento
Goal in Context	L'utente al sistema vuole lasciare un messaggio nella sezione commenti associata ad un'issue.
Preconditions	L'utente si è autenticato.
Success End Condition	L'utente ha scritto il messaggio e questo risulta presente nella sezione commenti relativa all'issue di riferimento.
Failed End Condition	Il messaggio non risulta presente nella sezione commenti della specifica issue.
Primary Actor	Utente

Trigger	L'utente richiede di lasciare un messaggio nella sezione commenti di una specifica issue.			
Main Scenario	Step n.	Utente	Actor 2//	System
	1	L'utente clicca il riquadro di testo in cui inserire il commento.		
	2	Scriva il commento relativa alla issue.		
	3	Clicca il tasto di invio al sistema.		
	4			Il Sistema riceve il messaggio.
	5			Aggiorna la sezione commenti della issue specificata dall'utente aggiungendo il suo commento.
	6	L'utente visualizza sullo schermo il proprio commento nella sezione commenti.		
	7			
	8			
	9			

USE CASE #2	Visualizza Issue
Goal in Context	L'utente vuole visualizzare tutte le issue presenti al momento.
Preconditions	L'utente si è autenticato.

Success End Condition	L'utente clicca il pulsante di visualizzazione delle issue e viene reindirizzato ad una schermata con tutte le issue presenti al momento.			
Failed End Condition	Le issue presenti nel Database non vengono visualizzate dall'utente. Il pulsante non reindirizza l'utente alla schermata delle issue.			
Primary Actor	UtenteReadOnly			
Trigger	L'utente clicca il pulsante di visualizzazione delle issue.			
Main Scenario	Step n.	UtenteReadOnly	Actor 2//	System
	1	L'utente clicca il pulsante di visualizzazione delle issue.		
	2			Il sistema reindirizza l'utente dalla dashboard alla schermata delle issue.
	3	L'utente visualizza tutte le issue presenti.		
	4			

1.6.2. Mock-Up



2. Documento di Design del Sistema

2.1. Architettura Backend

Il Backend consiste in classi Java servite mediante il framework Spring Boot. Si suddivide nelle cartelle:

- **config** = configurazione dell'applicazione: **SecurityConfig** (regole di sicurezza), **WebMvcConfig** (configurazione MVC, es. esposizione file statici), **AdminSeeder** (creazione utente admin all'avvio).
- **controller** = Il livello REST che riceve le richieste HTTP ed espone gli endpoint: **AuthController**, **IssueController**, **AdminController**, più **GlobalExceptionHandler** che intercetta le eccezioni e le trasforma in risposte HTTP coerenti.
- **dto** = Gli oggetti usati per il trasferimento dati tra client e server (richieste in ingresso e risposte in uscita), separati dalle entità di persistenza. È a sua volta suddiviso in sotto-package per area (dto/issue, dto/user, dto/comment, dto/report), così i DTO specifici di ciascuna risorsa restano raggruppati insieme.

- **exception** = Le eccezioni di dominio personalizzate (**ResourceNotFoundException**, **InvalidCredentialsException**, **EmailAlreadyInUseException**, **InvalidImageException**), pensate per essere catturate centralmente dal **GlobalExceptionHandler**.
- **model** = Le entità JPA persistite nel database (**User**, **Issue**, **Comment**) e gli enum di dominio associati (**Role**, **IssueType**, **IssueStatus**, **IssuePriority**).
- **repository** = Le interfacce Spring Data JPA per l'accesso ai dati (**UserRepository**, **IssueRepository**, **CommentRepository**) e la logica di filtri dinamici (**IssueSpecification**).
- **security** = Il meccanismo di autenticazione JWT vero e proprio: **JwtService** (generazione/validazione token) e **JwtAuthenticationFilter** (il filtro che intercetta ogni richiesta).
- **service** = La logica di business che non appartiene né al controller né al repository: come **ReportService** (calcolo dei report mensili).

2.1.1. Java

Si è scelto di utilizzare Java come linguaggio di programmazione del backend non solo in conformità al percorso di studio da noi intrapreso, ma anche perché esso permette di avere un sistema sicuro e con ottime prestazioni in fasi di esecuzione.

Java soddisfa pienamente il requisito di un linguaggio di programmazione **object-oriented** e la sua tipizzazione statica forte riduce il numero di errori riscontrati a runtime, soprattutto su un dominio con molte regole di business (ruoli, stati, permessi)

2.1.2. Spring Boot

Utilizzando Java come linguaggio di programmazione, ci è risultato semplice scegliere come framework Spring Boot, in quanto nativamente supportato dall'IDE IntelliJ IDEA utilizzato durante lo sviluppo e ricco di funzionalità che hanno facilitato la creazione del progetto:

- Fornisce **Dependency Injection**, che nel codice si traduce in classi come **SecurityConfig**, **JwtService**, **LocalImageStorageService**, etc, iniettate via costruttore. Questo comporta un maggiore disaccoppiamento dei componenti, essenziale per un pieno isolamento in fase di test
- Riduce il codice **boilerplate** mediante il gran numero di librerie, moduli e direttive disponibili.
- Espone nativamente **REST**

2.1.3. PostgreSQL

PostgreSQL è un database relazionale particolarmente popolare, non solo è open-source e gratuito, rispetta appieno le proprietà ACID e supporta nativamente vincoli dichiarativi e Spring Data JPA/Hibernate.

2.1.4. Spring Data JPA, Hibernate, Specification API

Questi tre componenti sono il cuore dell'interazione tra il backend e il database relazionale. Permettono di facilitare enormemente l'interazione con PostgreSQL.

- **Hibernate:** si tratta di un ORM (Object Relational Mapping) che traduce gli oggetti e classi Java in righe di tabelle relazionali e viceversa.
Utilizziamo Hibernate nella definizione dei models, ovvero classi che rappresentano le entità coinvolte nell'applicativo (Utenti, Issue, Commenti). Tra le fondamentali direttive di Hibernate, ricordiamo:
 - 1) **@Entity:** associa una classe ad una tabella del database
 - 2) **@Table:** permette di definire il nome della tabella
 - 3) **@Id:** indica la chiave primaria della tabella
 - 4) **@GeneratedValue:** indica come viene creata la chiave primaria (IDENTITY = auto-increment)
 - 5) **@Column:** permette di specificare delle proprietà per una colonna, come la possibilità di essere nulla o meno
 - 6) **@Enumerated:** permette l'uso di enum per un campo della tabella.
 - 7) Relazioni tra entità (**@OneToMany**, **@ManyToOne**, etc)
 - 8) **@JoinColumn:** specifica la foreign key
- **Spring Data JPA (JPA = Jakarta Persistence API):** si tratta di un modulo di Spring il cui scopo è eliminare il codice boilerplate per le tipiche operazioni CRUD (Create, Read, Update, Delete) effettuate su un database. Permette infatti di creare delle interfacce (delle Repository) che estendono JpaRepository e che contengono metodi il cui nome viene automaticamente associato ad una query sql. In sintesi, questo modulo ci ha permesso di effettuare query al database senza mai scrivere sql grezzo all'interno del codice.
- **Specification API:** tuttavia, anche uno strumento come Spring Data JPA ha i suoi limiti, e non permette di effettuare query particolarmente complesse. Ma la Specification API risolve questo problema, permettendo di definire delle condizioni, poi concatenabili con .where() e .and().

2.1.5. Lombok

Lombok è una libreria Java che riduce il codice boilerplate usando annotazioni che vengono elaborate in fase di compilazione.

Invece di scrivere manualmente getter, setter, costruttori, toString(), equals()/hashCode(), possiamo lasciare a Lombok il compito di generare automaticamente questo codice a partire da semplici annotazioni.

Principali annotazioni:

- **@Getter / @Setter:** generano getter e setter per i campi
- **@NoArgsConstructor / @AllArgsConstructor / @RequiredArgsConstructor:** generano costruttori

2.1.6. Rest API

Una **REST API** (Representational State Transfer) è uno stile architetturale per esporre risorse (in questo caso: utenti, issue, commenti, report) tramite endpoint HTTP, ognuno identificato da un URL e associato a un verbo HTTP che ne descrive l'operazione (GET per leggere, POST per creare, PUT per modificare).

Il backend usa **Spring Web MVC** con l'annotazione **@RestController**, che dice a Spring di serializzare automaticamente in JSON tutto ciò che i metodi restituiscono.

Abbiamo utilizzato REST in quanto è lo standard de facto per comunicazione tra un frontend SPA (React, in questo progetto) e un backend: è stateless, basato su JSON, facilmente consumabile con fetch.

Inoltre, essendo stateless per natura, si sposa perfettamente con un meccanismo di autenticazione anch'esso stateless come JWT: niente sessioni server-side da gestire o sincronizzare.

2.1.7. Spring Security

Spring Security è il framework che intercetta ogni richiesta HTTP in ingresso e decide se può proseguire verso il controller, in base a: chi è l'utente (autenticazione) e cosa può fare (autorizzazione).

La configurazione centrale è in **SecurityConfig**.

Abbiamo utilizzato Spring Security in quanto è un framework standard e consolidato nell'ecosistema Spring, testato in produzione su larga scala.

Inoltre permette di creare un modello dichiarativo (**authorizeHttpRequests**) che centralizza le policy di accesso in un unico punto leggibile, invece di controlli sparsi nei controller.

Il **SecurityContextHolder** offre un modo standard e thread-safe per sapere "chi sta chiamando" una funzione ovunque nel codice, senza passare l'utente manualmente tra i livelli.

2.1.8. JWT

Un **JWT** è un token firmato digitalmente che contiene un insieme di informazioni ("claims") sull'utente, in un formato compatto (header.payload.signature, codificato in Base64Url). Chi lo riceve può verificarne l'integrità (nessuno l'ha modificato) e l'autenticità (è stato emesso da chi conosce la chiave segreta), senza dover consultare un database o uno storage di sessioni.

Questa proprietà, verificabilità senza stato condiviso, è esattamente ciò che serve per un backend stateless come impostato in SecurityConfig.

Dunque, JWT è:

- Coerente con lo stile stateless: nessuna HttpSession da mantenere o condividere.
- Autosufficiente: contiene già ruolo e id utente come claim firmati, evitando una query al database ad ogni richiesta solo per sapere chi è l'utente.
- Naturale in un'architettura con frontend e backend su origini diverse: un header Authorization è più semplice da gestire di cookie di sessione cross-origin.
- Scadenza esplicita (24h) senza bisogno di un meccanismo di invalidazione lato server.

2.2. Architettura Front-end

Il frontend del progetto è una **SPA** (Single Page Application) separata dal backend, servita in sviluppo da Vite su <http://localhost:5173>.

Il frontend è costituito da una suddivisione in file .css (gestiscono lo stile e aspetto grafico del progetto) e file .tsx (un file che permette di scrivere typescript e html insieme), oltre a classici file TypeScript .ts per i service e interfacce.

Nel dettaglio, il frontend si suddivide nelle cartelle:

- **config** = indica l'indirizzo per comunicare con il backend
- **model** = modelli e interfacce che rispecchiano quelle situate nel backend
- **pages** = pagine visualizzabili dall'utente, costituite da file .tsx e .css
- **services** = servizi che interagiscono con il backend per ricevere informazioni o salvare/modificare oggetti.

2.2.1. TypeScript

TypeScript è un soprainsieme di JavaScript che aggiunge tipizzazione statica, verificata a compile-time.

In un progetto con un backend fortemente tipizzato (Java), usare JavaScript puro lato frontend introdurrebbe un disallineamento: gli stessi concetti (ruolo utente, forma di una issue, risposta di login) dovrebbero essere tenuti sincronizzati "a occhio" tra i due lati, senza alcun controllo automatico. TypeScript colma questo divario mediante la sua tipizzazione.

2.2.2. React + React Router

React è un framework frontend (o come affermano i suoi sviluppatori, una libreria) per costruire interfacce utente.

La UI del progetto cambia in modo sostanziale in base a chi è loggato (un admin vede pulsanti che un utente readonly non vede affatto) e a quale pagina è attiva. Un modello a componenti con stato e rendering dichiarativo si adatta naturalmente a questo tipo di UI, evitando la gestione manuale e imperativa del DOM che sarebbe necessaria con JavaScript. React è inoltre lo standard di fatto per SPA.

React Router è una libreria che aggiunge a una SPA React la navigazione tra pagine diverse, gestendo l'URL del browser senza però ricaricare l'intera applicazione dal server ad ogni cambio pagina (comportamento tipico di una SPA, diverso da un sito multi-pagina tradizionale).

Senza di esso, un'app React a singola pagina non avrebbe URL distinti per le diverse viste, perdendo funzionalità basilari del browser come il tasto indietro o i link condivisibili.

2.2.3. Vite

Vite è uno strumento di build e server di sviluppo per progetti frontend, pensato per essere molto rapido. Nel progetto è configurato in modo minimale, per abilitare il Fast Refresh (l'aggiornamento dei componenti React durante lo sviluppo).

È stato scelto in quanto è lo strumento consigliato dalla stessa documentazione React per nuovi progetti, con configurazione minima necessaria per ottenere TypeScript e un dev server funzionanti insieme.

3. Documento di Design del Software

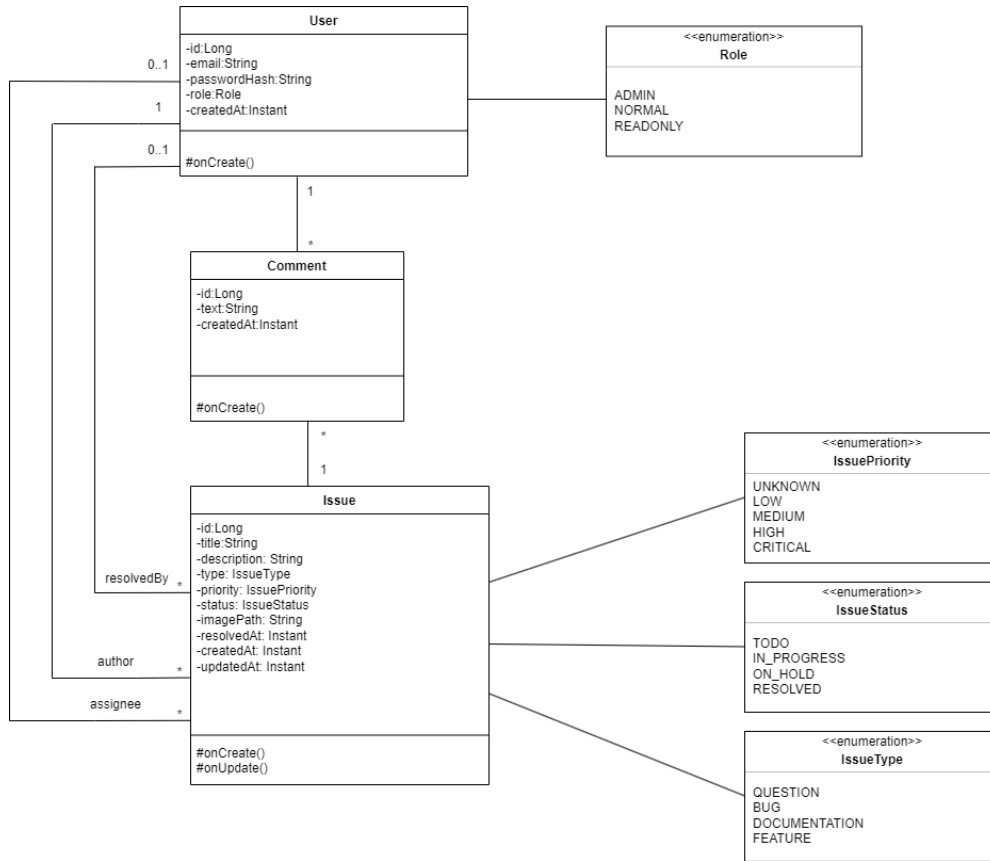
3.1. Link GitHub

<https://github.com/Antonio-Pocomento/BugBoard26>

Mediante la pagina è possibile visualizzare anche report con statistiche come numero di commit e di contributors, frequenza dei commit, etc, nella sezione "Insights".

Il progetto è stato sviluppato mediante la funzionalità "Code with Me" di IntelliJ IDEA, che permette a più individui di lavorare sullo stesso codice anche se non presente fisicamente sui loro dispositivi. Per questo motivo, per evitare anche possibili conflitti o problemi, le commit sono state effettuate sempre dal membro del gruppo che attivava la suddetta funzionalità.

3.2. Schema per la Persistenza dei Dati



3.3. Design Patterns

3.3.1. Dependency Injection

È il Design Pattern più presente all'interno del progetto:

```
public IssueController(IssueRepository issueRepository, UserRepository
userRepository, CommentRepository commentRepository, ImageStorageService
imageStorageService)
```

Anche se è il framework ad applicarlo, ogni controller/servizio riceve le proprie dipendenze via **constructor injection**.

3.3.2. Altri Pattern

Anche se non **GoF** (Gang of Four pattern), altri stili architetturali utilizzati nello sviluppo sono:

- **Repository pattern** = astrazione dell'accesso ai dati.
- **Specification pattern** = costruisce criteri di query componibili.
- **DTO pattern** = netta separazione tra entità JPA (Issue, User) e DTO di request/response (IssueResponse, CreateIssueRequest, ecc.).

Il progetto segue soprattutto le convenzioni di Spring Boot (repository, DTO, controller) piuttosto che pattern GoF implementati esplicitamente. Tra quelli studiati, l'unico particolarmente presente è il Dependency Injection; tutti gli altri (Singleton, Factory, Builder, Composite, Decorator, Observer, State) non sono implementati nel codice, sebbene alcuni compaiano nelle librerie usate.

4. Attività di Testing

4.1. Tecnologie Utilizzate

4.1.1. JUnit

I test sono stati creati utilizzando JUnit 5, un popolare framework di unit testing per il linguaggio Java.

Le istruzioni principali utilizzate nei file:

- **@Test**: segna un metodo come caso di test. JUnit crea una nuova istanza della classe di test per ogni metodo (non riusa la stessa istanza), proprio per garantire che i test siano isolati tra loro.
- **@DisplayName("...")**: un nome leggibile mostrato nei report al posto del nome del metodo Java.
- **@BeforeEach** / **@AfterEach**: eseguiti automaticamente prima/dopo ogni test. Nel `setUp()` configuriamo gli stub comuni (es. l'encoder che restituisce sempre "hashed-password"); nel `tearDown()` puliamo il `SecurityContextHolder` perché è uno stato statico condiviso e, senza pulirlo, un test potrebbe "vedere" l'utente autenticato da un test precedente.
- **@ExtendWith(MockitoExtension.class)**: è il punto di aggancio tra JUnit e Mockito. JUnit 5 non ha il mocking incorporato: il suo extension model permette a librerie esterne di agganciarsi al ciclo di vita del test tramite dei "callback" (`BeforeEachCallback`, ecc.). MockitoExtension usa proprio questo meccanismo per inizializzare i mock prima di ogni test.

4.1.2. Mockito

Accennato nel precedente paragrafo, Mockito è un popolare framework di mocking open-source per il linguaggio Java: Mockito permette di creare oggetti fittizi, chiamati mock, per simulare il comportamento di dipendenze complesse o esterne (come database, servizi web o API di rete).

Le istruzioni principali utilizzate nei file:

- **@Mock**: crea un oggetto finto al posto di `UserRepository`, `PasswordEncoder`, ecc. Di default ogni metodo del mock, se non configurato, restituisce null (o 0/false/liste vuote per i tipi primitivi).

- **@InjectMocks**: crea un'istanza vera di AdminController e ci inietta dentro i mock sopra, cercando un costruttore compatibile.
- **when(...).thenReturn(...)**: la sintassi di stubbing, quando (when) viene chiamato il metodo, allora restituisce (thenReturn) un oggetto da noi deciso.
- **when(...).thenAnswer(inv -> {...})**: come sopra ma con logica personalizzata: nel nostro setUp() lo usiamo per simulare il comportamento del DB, assegnando un id e un createdAt all'utente "salvato".
- **verify(mock).metodo(...)**: dopo l'esecuzione, controlla che una certa chiamata sia effettivamente avvenuta.
- **verify(mock, never())**: controlla l'opposto: che non sia mai stata chiamata (utile per dimostrare che, se l'autorizzazione fallisce, save() non viene mai invocato).
- **ArgumentCaptor<T>**: "cattura" l'oggetto passato a un mock durante l'esecuzione, per potergli fare asserzioni dopo. Lo usiamo per prendere l'oggetto User/Comment passato a .save(...) e verificare che i suoi campi siano corretti.
- **any(), anyString()**: matcher generici.
- **lenient()**: disattiva il controllo "stubbing non necessario" di Mockito per quella riga. Normalmente Mockito lancia un errore se configuriamo when(...) su qualcosa che poi un test non arriva mai a chiamare.
- **mock(SecurityContext.class)**: la versione "manuale" di @Mock, usata per creare un mock al volo dentro un metodo helper invece che come campo della classe.

4.1.3. AssertJ

È la libreria che usiamo per le asserzioni finali, un'alternativa agli assertEquals(...) nativi di JUnit.

- **assertThat(valore).isEqualTo(atteso)**: assertThat restituisce un oggetto "wrapper" specifico per il tipo (per una stringa un StringAssert, per un oggetto un ObjectAssert, ecc.), che espone metodi di verifica concatenabili (.isEqualTo(...), .isNull(), .isNotNull(...)). Il vantaggio principale è che i messaggi di errore in caso di fallimento sono molto più chiari delle informazioni fornite dagli assert di JUnit.

- **assertThatThrownBy()** -> **metodo(...)**: prende una lambda, la esegue, cattura l'eccezione lanciata e permette di fare asserzioni su di essa: `.isInstanceOf(AccessDeniedException.class)`.

4.2. Test e Classi di Equivalenza

4.2.1. `createUser(createUserRequest request)`

Il metodo presenta due punti di decisione rilevanti a livello di logica: l'esistenza pregressa dell'email (if (...isPresent())) e il valore del ruolo assegnato.

Classe di equivalenza	Valida / non valida	Test
Email non presente nel repository	Valida	<code>createUserWithNormalRole / createUserWithAdminRole / createUserWithReadonlyRole</code>
Email già presente nel repository	Non valida → <code>EmailAlreadyInUseException</code>	<code>emailAlreadyInUse</code>
Ruolo = NORMAL	Valida	<code>createUserWithNormalRole</code>
Ruolo = ADMIN	Valida	<code>createUserWithAdminRole</code>
Ruolo = READONLY	Valida	<code>createUserWithReadonlyRole</code>
Password in chiaro → hash memorizzato	Valida (verifica indiretta)	<code>savedUserHasHashedPasswordAndCorrectFields</code>

Copertura raggiunta: entrambi i rami dell'unico if del metodo (email libera / email già in uso) sono coperti, così come tutti e tre i valori dell'enumerazione Role, ottenendo copertura di decisione (branch coverage) del 100% per la logica del metodo.

4.2.2. createComment (Long id, CreateCommentRequest request)

Il metodo ha due diramazioni (esistenza della issue e ruolo readonly dell'utente). La suite di test riflette questa scelta di design verificando esplicitamente che ruoli diversi (assegnatario, admin, utente estraneo, readonly) producano tutti esito positivo (con un'eccezione nel caso readonly)

Classe di equivalenza	Valida / non valida	Test
Issue esistente (findByld → presente)	Valida	assigneeCanAddComment, adminCanAddComment, unrelatedUserCanAddComment, savedCommentHasCorrectIssueAndAuthor
Issue inesistente (findByld → assente)	Non valida → ResourceNotFoundException	noIssueFound
Autore = assegnatario della issue	Valida	assigneeCanAddComment
Autore = admin	Valida	adminCanAddComment
Autore = utente estraneo alla issue	Valida (nessuna restrizione prevista)	unrelatedUserCanAddComment
Autore = utente readonly	Non valida → AccessDeniedException	readonlyUserCantAddComment

Copertura raggiunta: entrambi gli esiti del predicato sull'esistenza della issue sono coperti, oltre ai diversi ruoli degli utenti, ottenendo il 100% di copertura di decisione per il metodo; il test savedCommentHasCorrectIssueAndAuthor completa la verifica funzionale controllando, tramite ArgumentCaptor, che l'entità Comment riporti correttamente issue e author.

4.2.3. changelssue(Long id, ChangelssueRequest request)

È il metodo con il maggior numero di diramazioni indipendenti.

Classe di equivalenza	Valida / non valida	Test
Richiedente con ruolo ADMIN	Valida, sempre autorizzato	adminCanModifyAnyIssue
Richiedente NORMAL, assegnatario della issue	Valida, autorizzato	assigneeCanModifyAssignedIssue
Richiedente NORMAL, non assegnatario	Non valida → AccessDeniedException	unauthorizedUserCantChangelssue
Issue con id esistente	Valida	tutti i test tranne noIssueFound
Issue con id inesistente	Non valida → ResourceNotFoundException	noIssueFound
Transizione stato ≠ RESOLVED → RESOLVED	Valida, imposta resolvedAt / resolvedBy	changingToResolvedSetsTimestamps
Transizione RESOLVED → stato ≠ RESOLVED	Valida, azzera resolvedAt / resolvedBy	changingFromResolvedResetsTimestamps
Transizione RESOLVED → RESOLVED (invariato)	Valida, non sovrascrive il timestamp	resolvedIntoResolvedNoOverwriteTimestamp
assigneeEmail assente (null)	Valida, nessuna modifica all'assegnatario	assigneeCanModifyAssignedIssue, adminCanModifyAnyIssue, ecc.

Classe di equivalenza	Valida / non valida	Test
assigneeEmail = stringa vuota	Valida, rimozione dell'assegnatario	adminCanRemoveAssignee
assigneeEmail = email di un utente esistente	Valida, riassegnazione	adminCanChangeAssignee
assigneeEmail = email senza utente corrispondente	Non valida → IllegalArgumentException	invalidAssigneeEmail
Modifica di assigneeEmail richiesta da ADMIN	Valida, consentita	adminCanChangeAssignee, adminCanRemoveAssignee
Modifica di assigneeEmail richiesta da non ADMIN	Non valida → AccessDeniedException	assigneeCantChangeltsself
Richiedente con ruolo READONLY	Non valida → AccessDeniedException	readonlyUserCantChangelssue
Richiedente con ruolo READONLY e assegnatario	Non valida → AccessDeniedException	readonlyAssigneeCantChangelssue
assigneeEmail = email di un utente readonly	Non valida → IllegalArgumentException	adminCantAssignIssueToReadOnlyUser

4.3. Limiti Individuati e Possibili Estensioni

- Trattandosi di unit test a livello di controller con contesto Spring non avviato, i vincoli di Bean Validation (@NotBlank, @Email, @Size) sulle DTO CreateUserRequest, ChangelssueRequest e CreateCommentRequest non vengono esercitati.
- Non sono coperti da questi unit test eventuali vincoli a livello di persistenza (es. vincolo di unicità sull'email a livello di database), che richiedono test di integrazione con un database reale o in-memory.